Performance and dependability evaluations of two solutions implementing a publisher-subscriber primitive.

Marco Alberti, Cristian Andrei Mai Mihai

September 2024

Abstract

With the desire to delve deeper into the "Dependable Distributed Systems" subject of our Computer Engineering course, we decided to accompany the achievement of the exam with the creation of a project on one of the topics covered during the subject itself. This report will focus on the study and the performance and dependability evaluations of two solutions that implement the publisher-subscriber paradigm, RabbitMQ and ZeroMQ, by doing experiments on our implemented system that uses them. We will start with a brief description of the implementation of the system, moving on to the definition of the workload, the experiments carried out for the evaluations themselves, and then end with the final considerations and a short tutorial on how to start the project on your computer, for those interested.

1 Introduction

A **distributed system** is a collection of physically separated autonomous devices that communicate by coordinating and sharing whatever resource. The devices have a common goal and are connected by a centralized computer network that is equipped with distributed system software, appearing as a single computing system to final users.

A distributed system is **dependable** when it is committed to avoid service failures that are frequent and intense in a way that is not acceptable. One of the main topics dealing with information diffusion in such large-scale systems, is the **publish–subscribe paradigm**.

The publish–subscribe paradigm. It is a communication (or messaging) paradigm where **publishers** produce messages, or generally speaking data, and **subscribers** declare interests with a sort of filter on the set of these published messages, called subscriptions. Furthermore, there is a mediator, called **Event Notification Service**, that notifies to each subscriber every message

that matches at least one of its subscriptions; it is contrasted to the typical communication paradigm model where publishers send messages directly to subscribers.

This paradigm is a perfect solution especially for problems like **many-to-many communications** and **space, time and synchronization decoupling**.[1]

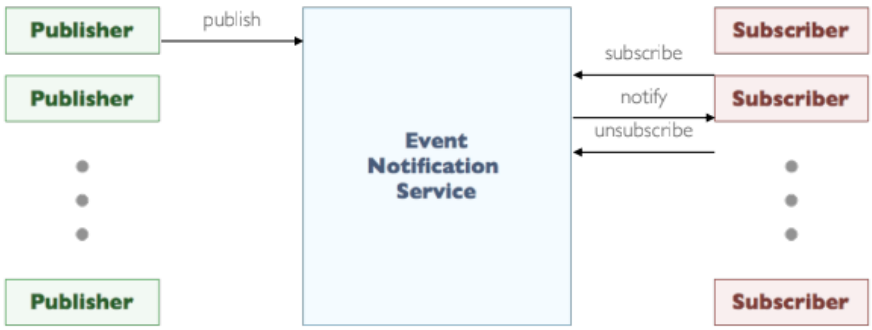

Figure 1: General structure of the Publisher-Subscriber paradigm.

2 Implementation of the environment

Given the premises, we will now start talking about the implementation of our distributed system. We decided to use the following technologies:

- **RabbitMQ** that is a message-oriented middleware, in particular a messaging and streaming broker, that implements the AMQP[2] protocol;
- **ZeroMQ** that is an asynchronous networking (or messaging) library, intended for use in distributed or concurrent applications. It provides a message queue, but unlike message-oriented middleware[3], a ZeroMQ system can operate without a dedicated message broker;
- **Python** programming language, since there are numerous libraries that allow us to abstract some problems to solve, it is very widespread and suitable for developing distributed applications and has an easy syntax;
- **Ubuntu** 22.04 OS. We will use Linux, because it is the operating system used by most servers;
- a **local machine** with a CPU with 16 core.

[1] Figure 1 and some definitions taken from the university course material.
[2] Advanced Message Queuing Protocol.
[3] An intermediary software that facilitates communication and connectivity between various applications and other components of the distributed system that are often very different from each other.

A brief explanation of the system. We implemented two **modes** in which the distributed system can be run. The first one using RabbitMQ and the second one using ZeroMQ, so that we can compare them to understand the pros and cons of each.

Once the system is started in a mode, a fixed number of processes are generated that correspond to the number of publishers and subscribers and then the entire communication framework is put up, composed of the mediator (ENS), thanks to which it will be possible to exchange messages according to the publish-subscribe paradigm. The mediator is composed by a **cluster of 3 different brokers**: in the rabbitmq mode the cluster is pre-built as we will explain in section 4 and ready at system startup; in zeromq mode, the cluster is built after the system startup and before the start of communications, as it is a brokerless library.

The generation of messages by the publishers is **random** and follows an exponential probability distribution. The subscribers will subscribe to the type of data they want to receive through **topic-type subscriptions**. Even the topics that the nodes want to send or receive are generated randomly and in the various possible cases, there is also the one for which **fanout exchange** is created, i.e. that in which a certain message is sent to everyone without any kind of filter.

Finally, at runtime there is a certain probability for which a publisher or a subscriber can crash or voluntary exit or join the system.

3 Evaluation design and results collected

In this section of the report, we will talk about the main part of the project, namely the performance and dependability evaluations of the two previously reported solutions.

3.1 Workload

First of all, we define the system workload since the performance and reliability of the system depend a lot on it, in particular on the **arrival pattern** and the **service demands**. Therefore, the workload is nothing more than the set of all the inputs that the system receives in a certain period of time defined a priori, from its environment.

Functional characterization. The analysis is carried out from a point of view that is external to the system, so observing it from outside the system itself. In our case the most relevant parameters (for each request) are defined by the arrival of the messages, which form a component (the unique) of the workload, sent or received by the publishers or the subscribers respectively.

Each process is handled by about 1 core of the local machine CPU, simulating that each process in that distributed system is located in a physically separated node: this is possible because we modified the priorities of all the processes of the

local machine, giving greater priority to the processes of our distributed system so as not to incur multiple unwanted or unexpected CPU context switches.

The arrival pattern of the component. We calculated and collected these parameters, in a time interval of 60s:

- **number of arrivals**: the number of arrivals (sent messages) per publisher;
- **latencies**: the mean observed latency per subscriber, so the time occurred to a message to be received;
- **inter-arrival times** (regularity): time that passes between two message occurrences. We generated the estimation of the Probability Density Function related to that parameter.

Since the arrivals are random, we modeled them with a random variable.

3.2 Performance evaluation

Our implementation is not really the actual system since it is located in our single local machine, so we created a sort of simulation of it that models the main behaviours so that we can do experiments this way using the benchmarking technique.

We have passively monitored the system at work and obtained raw data, saving the logs and once we took the measures, we created 3 different bar charts with a table of measures. We calculated and collected these performance measures using as input those one collected to characterize the workload. We considered a system observation interval again equal to 60s, and all the below measures are taken wrt this interval:

- **number of completions**;
- **throughput**;
- **busy period of time**;
- **resource utilization**: gives an indication of the percentage of time the system is busy for a given workload;
- **average service time per completion**.

.

Experiments conducted. We decided to make 12 experiments per mode, so we have done 24 different executions of the simulation using combinations of theses options in both the two modes:

- generation of 7 publishers and 7 subscribers;

- generation of 7 publishers and 1 subscriber and 1 topic available to be used;
- generation of 1 publisher and 7 subscribers and 1 topic available to be used;
- no simulated crashes;
- simulated crashes for publishers and subscribers but not for brokers (in the cluster);
- simulated crashes for publishers and subscribers with a crash of 1 broker (in the cluster) after 15 seconds since the starting time;
- simulated crashes for publishers and subscribers with a crash of 2 brokers (in the cluster) after 15 and 16 seconds respectively.

By observing all the data, we chose the most interesting ones to mention here in this report:

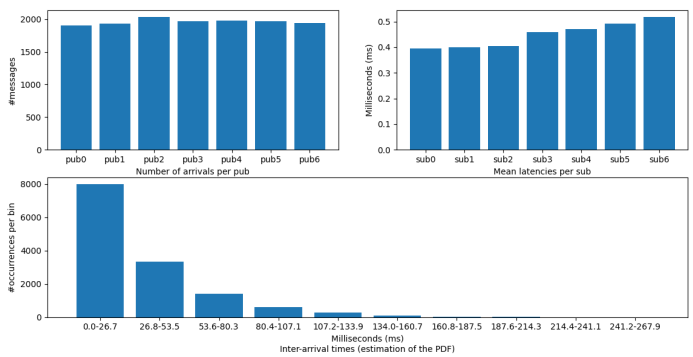

Figure 2: Rabbitmq mode; 7 pub, 7sub; either new runtime creations, nor crashes.

```
-----------Information obtained by the simulation-----------
System observation interval: 60s
Number of arrivals:  13729
Mean latency of the system:  0.449ms
Number of completions:  48434
Lost messages: 0
Arrival rate: 228.816 msgs/s
Throughput: 807.233 completions/s
Busy period of time: ~60s
Resource utilization: ~1ms
Average service time per completion: 0.001ms
------------------------------------------------------------
```

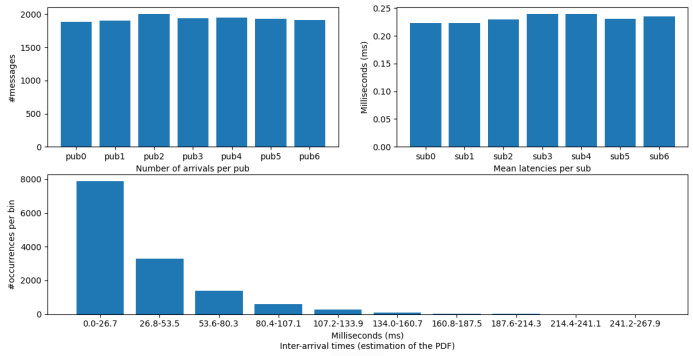

Figure 3: Zeromq mode; 7 pub, 7sub; either new runtime creations, nor crashes.

```
-----------Information obtained by the simulation-----------
System observation interval: 60s
Number of arrivals:  13536
Mean latency of the system:  0.218ms
Number of completions:  111063
Lost messages: 0
Arrival rate: 225.6 msgs/s
Throughput: 1851.05 completions/s
Busy period of time: ~60s
Resource utilization: ~1ms
Average service time per completion: 0.001ms
------------------------------------------------------------
```

The first experiment we mention is done with a creation of 7 publishers and 7 subscribers without introducing at runtime other nodes or crashes: we can infer by the charts and collected simulation results that the main difference between them wrt performances is the latency, so the ZeroMQ library is faster than RabbitMQ Broker. This is confirmed even in the cases when we start the system with 1 publisher and 7 subscribers (or vice versa), infact:

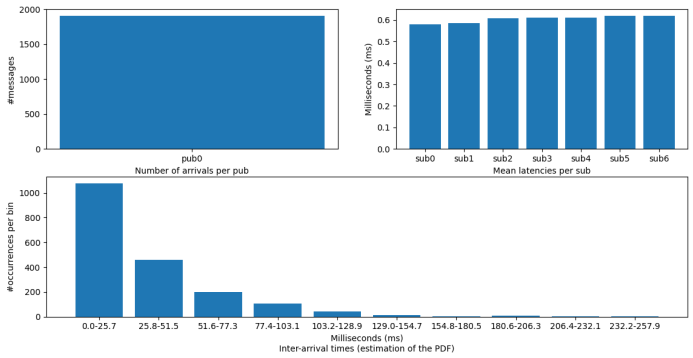

Figure 4: Rabbitmq mode; 7 pubs, 1 sub; either new runtime creations, nor crashes; 1 only available topic.

```
-----------Information obtained by the simulation-----------
System observation interval: 60s
Number of arrivals:  1907
Mean latency of the system:  0.605ms
Number of completions:  13349
Lost messages: 0
Arrival rate: 31.783 msgs/s
Throughput: 222.483 completions/s
Busy period of time: ~60s
Resource utilization: ~1ms
Average service time per completion: 0.004ms
------------------------------------------------------------
```

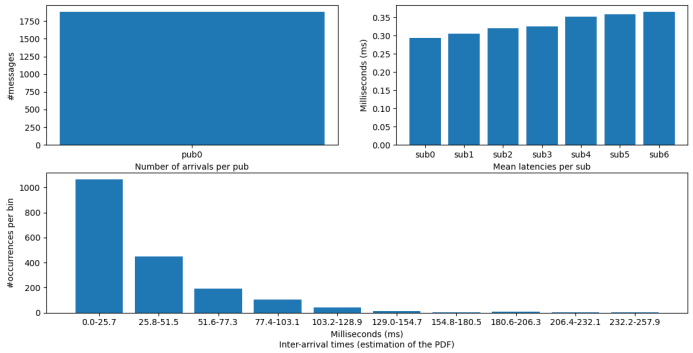

Figure 5: Zeromq mode; 7 pubs, 1 sub; either new runtime creations, nor crashes; 1 only available topic.

```
-----------Information obtained by the simulation-----------
System observation interval: 60s
Number of arrivals:  1882
Mean latency of the system:  0.331ms
Number of completions:  39522
Lost messages: 0
Arrival rate 31.367 msgs/s
Throughput: 658.7 completions/s
Busy period of time: ~60s
Resource utilization: ~1ms
Average service time per completion: 0.002ms
------------------------------------------------------------
```

Furthermore, it is important to note that, if the only publisher in the system were to crash, the system would continue to function but the system work would go back to being null.

Finally, we consider a last experiment:

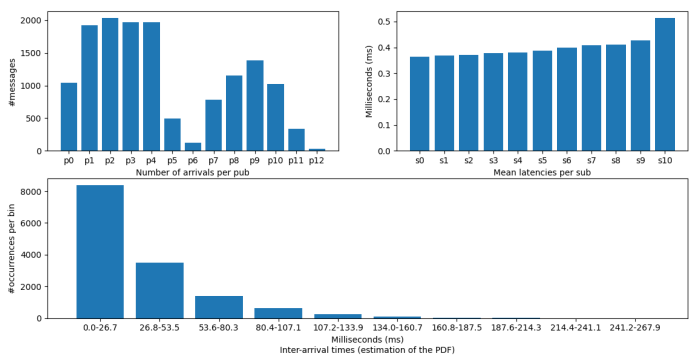

Figure 6: Rabbitmq mode; 7 pub, 7sub; 2 broker crashes.

```
-----------Information obtained by the simulation-----------
System observation interval: 60s
Number of arrivals:  14282
Mean latency of the system:  0.400ms
Number of completions:  55194
Lost messages: 6
Arrival rate: 238.033 msgs/s
Throughput: 919.9 completions/s
Busy period of time: ~60s
Resource utilization: ~1ms
Average service time per completion: 0.001ms
------------------------------------------------------------
```

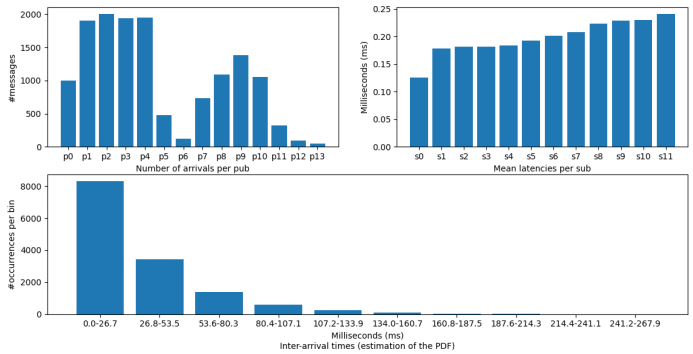

Figure 7: Zeromq mode; 7 pub, 7sub; 2 broker crashes.

```
-----------Information obtained by the simulation-----------
System observation interval: 60s
Number of arrivals:  14129
Mean latency of the system:  0.198ms
Number of completions:  142386
Lost messages: 0
Arrival rate: 235.483 msgs/s
Throughput: 2373.1 completions/s
Busy period of time: ~60s
Resource utilization: ~1ms
Average service time per completion: 0.0004ms
------------------------------------------------------------
```

This other experiment we want to mention is done with a creation of 7 publishers and 7 subscribers with a chain crashes of two brokers in the cluster, within 1 second of each other. In this case we can note, in addition to the fact of latencies seen previously, in rabbitmq mode there is a loss of 6 messages, 2 losses by sub2, 2 losses by sub3 and 2 losses by sub4.

3.3 Dependabiliy evaluation

We augmented **fault tolerance** through redundancy in software, namely creating as we said a cluster of 3 brokers. In this way all the publishers and subscribers had the possibility, in the event of a crash of the main server, to connect to one of the other two remaining and communicate anyway. Returning to an experiment shown previously, in the case of simulating a multiple broker crash in close proximity, ZeroMQ turns out to be more fault tolerant than RabbitMQ, as the latter does not fully guarantee that the messages sent are all received. We can infer that redundancy alone does not guarantee a higher level of robustness besides dependability.

4 Conclusions

We conclude that RabbitMQ is more complex because has a lot functionalities such as monitoring utilities that collect some overhead and are resources consuming. On the other hand, ZeroMQ has better performances, infact it is no coincidence that it is a lightweight library.

4.1 Contributions

Regarding contributions, there is no clear separation of our contributions. After we figured out what to do, we divided all the work into small tasks and split them up. We have always worked closely together, both on their own and using the "Pair programming" technique, especially when problems needed to be resolved. Almost every day, we made a work call to discuss the tasks that needed to be done for the day.

How to run the system. In case you want to test the system on your Linux machine, these are the steps to follow to do so:

1. download and start the RabbitMQ server on your local machine following the instructions on this link, choosing the right version of your distribution;

2. download the necessary libraries and python3 to execute correctly the system, using the terminal and running these commands:

```
> sudo apt update
> sudo apt install git python3 python3-pip
> pip install matplotlib pika zmq
```

3. using the terminal, enable the management plugin and create the cluster of 3 RabbitMQ server instances on the same local machine:

```
> sudo rabbitmq-plugins enable rabbitmq_management

> sudo RABBITMQ_NODE_PORT=5673 RABBITMQ_SERVER_START_ARGS=
"-rabbitmq_management listener [{port,15673}]" RABBITMQ_NODENAME=
rabbit2 rabbitmq-server -detached
> sudo RABBITMQ_NODE_PORT=5674 RABBITMQ_SERVER_START_ARGS=
"-rabbitmq_management listener [{port,15674}]" RABBITMQ_NODENAME=
rabbit3 rabbitmq-server -detached

> sudo rabbitmqctl -n rabbit2 stop_app
> sudo rabbitmqctl -n rabbit2 join_cluster rabbit@<your_hostname>
> sudo rabbitmqctl -n rabbit2 start_app

> sudo rabbitmqctl -n rabbit3 stop_app
> sudo rabbitmqctl -n rabbit3 join_cluster rabbit@<your_hostname>
> sudo rabbitmqctl -n rabbit3 start_app
```

obtaining a cluster with rabbit (listening on port 5672), rabbit2 (listening on port 5673) and rabbit3 (listening on port 5674) machines. The first one, is already created by the installation at point (1);

4. clone the repository where the source code of our project is present. It can be reached via this link:

```
> git clone <the_repo_link>
```

5. using the terminal, move inside the newly cloned repository and then you can start the system with the start.sh file, whose syntax is this:

```
> ./start.sh <rabbitmq|zeromq> <num_of_pubs> <num_of_subs>
```

For instance, you could do this:

```
> ./start.sh rabbitmq 2 5
```

that will start the system in rabbitmq mode and generate 2 publishers and 5 subscribers. If you want to start the system in zeromq mode, you will insert this option instead of rabbitmq, in the example command; the same thing goes for the number of publishers and subscribers. For a more accurate and closer execution to ours, you can give greater priority to the processes of the distributed system, compared to others in the local machine, by following this guide.

Thanks for reading.